

The Grokking Boundary:

Discrete Logarithm Hardness and Group Structure in Transformers

David Vesterlund

September 2026

Abstract

We investigate when and why neural networks can learn the discrete logarithm problem (DLP) through gradient-based training. Prior theoretical work has established that gradient-based methods cannot efficiently learn the parity bit of the discrete logarithm in cyclic groups of large prime order. We empirically map the boundary of this hardness result across model architectures, group structures, and bit sizes. Using a 199-experiment design on LUMI-G supercomputing infrastructure, we find: (1) MLPs can learn individual DLP parity bits for small primes ($b \leq 16$) through direct generalization, but fail for $b \geq 20$, confirming the predicted hardness threshold; (2) bit-tokenizer Transformers fail to grok the prime-order subgroup DLP at any tested bit size ($b = 7\text{--}26$), consistent with the hardness prediction; (3) the same Transformer architecture *does* grok the additive group (modular addition) at $b = 16$, but fails at $b \geq 18$, revealing that the grokking boundary is group-structure-dependent; and (4) across a 2×2 design varying base visibility (hidden vs. visible) and target granularity (parity vs. full log), no configuration produces grokking on prime-order DLP for $b = 7\text{--}14$ at 200k epochs. These results establish a sharp *grokking boundary* determined by the interaction of group structure and representation, not by model capacity alone.

1 Introduction

The *grokking* phenomenon (Power et al., 2022)—in which a neural network abruptly transitions from memorization to generalization long after overfitting—has been studied extensively on modular arithmetic tasks. A natural question is whether grokking extends to the discrete logarithm problem (DLP), a structured inverse problem on cyclic groups.

Takhanov et al. (2024) proved that gradient-based methods cannot efficiently learn the parity bit of the discrete logarithm in cyclic groups of large prime order. This hardness result raises a fundamental question: *where exactly is the boundary between learnable and unlearnable DLP instances?*

In this work, we map this boundary empirically. We do not attempt to violate the hardness theorem—our experiments use small primes and make no claims about cryptographic-scale DLP. Instead, we ask: at what scale, with which architecture, and for which group structure does DLP become resistant to gradient-based learning?

Contributions.

1. We replicate the gradient-concentration hardness prediction using MLPs on DLP parity bits, confirming a transition from direct generalization ($b \leq 16$) to memorization-only ($b \geq 20$).

2. We show that bit-tokenizer Transformers fail to grok the prime-order subgroup DLP at any tested bit size ($b = 7\text{--}26$), consistent with the hardness theorem.
3. We demonstrate that the *same* Transformer architecture groks the additive group (modular addition) at $b = 16$ but fails at $b \geq 18$, revealing a group-structure-dependent grokking boundary.
4. We conduct a 2×2 discovery experiment (hidden/visible base $\times$ parity/full-log target) across $b = 7\text{--}14$, finding no grokking in any configuration at 200k epochs.

2 Background and Related Work

Discrete logarithm problem. Given a generator g of a cyclic group G of order q and an element $h \in G$, the DLP asks for x such that $g^x \equiv h$. In $\mathbb{F}_p^*$, the group order is $q = p - 1$. We also consider the prime-order subgroup DLP, where g generates a subgroup of prime order q inside $\mathbb{F}_p^*$.

Gradient-concentration hardness. Takhanov et al. (2024) proved that for cyclic groups of prime order q , any gradient-based method requires exponentially many steps to learn the parity bit of the discrete logarithm from random examples. The proof relies on the concentration of the gradient signal: for prime-order groups, the gradient of the loss with respect to the parity bit is exponentially small in $\log q$.

Grokking. Grokking (Power et al., 2022) is a delayed generalization phenomenon where neural networks transition from memorization to generalization long after overfitting training data. It has been studied on modular addition (Nanda et al., 2023), multiplication, and permutation composition. Liu et al. (2022) developed an effective theory of representation learning during grokking, and Thilak et al. (2022) identified the slingshot mechanism as a driver of grokking dynamics.

DLP grokking. Recent work (Vesterlund, 2026) demonstrated that Transformers can grok the variable-base DLP in $\mathbb{F}_{113}^*$, with mechanistic analysis revealing alignment with multiplicative character structure. That work used a full-integer tokenizer and composite-order groups. The present study complements this by investigating when DLP *resists* grokking.

3 Experimental Design

We design a five-stage experimental program to isolate the factors determining DLP learnability. All experiments use PyTorch on LUMI-G (AMD MI250X GPUs).

3.1 Stage 0: MLP Replication

We replicate the gradient-concentration hardness prediction using a simple MLP architecture. The task is to learn a single parity bit of the discrete logarithm: given (g, h) where $g^x \equiv h \pmod{p}$, predict $\text{bit}_i(x)$ for bit position i .

Setup.

- Architecture: 2-layer MLP with 1M parameters, ReLU activation.
- Primes: p chosen so that $q = p - 1$ has $b \in \{16, 18, 20, 22, 24, 26\}$ bits.

- Training: 2000 epochs, AdamW, weight decay 0.01.
- Repetitions: 5 random initializations per bit size, 3 bit positions per repetition (15 runs per bit size, 90 total).

3.2 Stage 0b: Theorem Uniform Distribution

The hardness theorem assumes inputs drawn from the uniform distribution. We verify that the hardness prediction holds under this distribution.

Setup.

- Same MLP architecture as Stage 0. Bit sizes: $b \in \{16, 20, 24\}$, 3 seeds each (9 runs).
- Training: 2000 epochs, uniform input distribution.

3.3 Stage 1: Architecture Bridge

We test whether a bit-tokenizer Transformer—the architecture that groks DLP in prior work—can learn the prime-order subgroup DLP.

Setup.

- Architecture: 2-layer Transformer, $d_{\text{model}} = 128$, 4 heads, $\sim 425\text{k}$ parameters. Bit tokenizer: each integer is represented as a sequence of b bits.
- Task: prime-order subgroup DLP, q prime with $b \in \{16, 18, 20, 22, 24, 26\}$ bits.
- Training: 2000 epochs, 3 seeds per bit size (18 runs).

3.4 Stage 2: Grokking Bridge

We test whether the same bit-tokenizer Transformer can grok the *additive* group (modular addition), which has a simpler group structure.

Setup.

- Same Transformer architecture as Stage 1.
- Task: modular addition $(a, b) \mapsto (a + b) \bmod q$, q prime with $b \in \{16, 18, 20, 22, 24, 26\}$ bits.
- Training: 100k epochs (to allow grokking), 3 seeds per bit size (18 runs).

3.5 Stage 3: 2×2 Discovery

We conduct a 2×2 factorial design varying two factors:

- **Base visibility:** *hidden* (base g not given, model must infer) vs. *visible* (base g provided as input).
- **Target granularity:** *parity* (predict $x \bmod 2$) vs. *full* (predict full x).

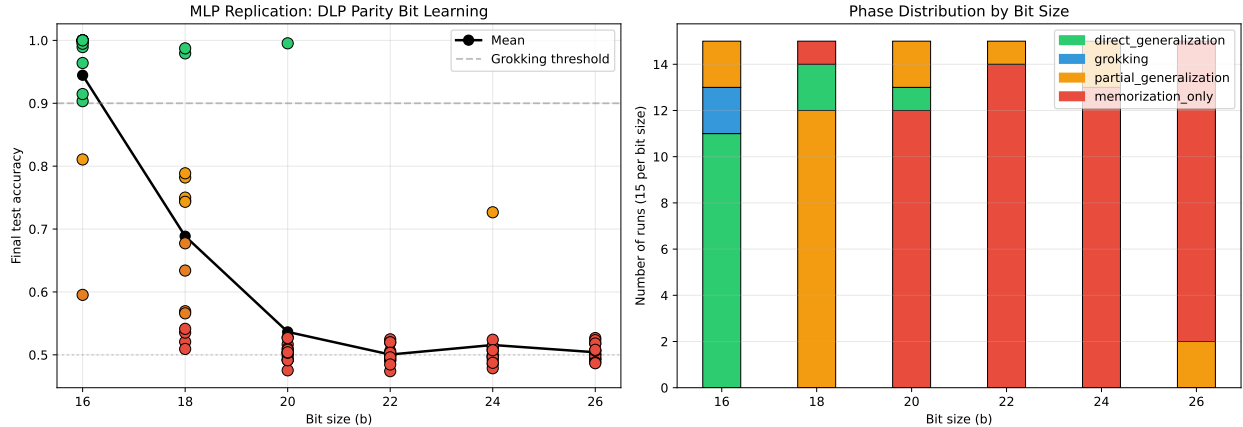


Figure 1: MLP replication of DLP parity-bit learning. **Left:** Final test accuracy for all 90 runs (15 per bit size). Green = direct generalization ($\geq 90\%$), orange = partial, red = memorization-only. **Right:** Phase distribution by bit size. The transition from direct generalization to memorization occurs between $b = 16$ and $b = 20$.

Table 1: Stage 0: MLP DLP parity-bit learning phase distribution.

Bit size	Direct gen.	Grokking	Partial gen.	Memorization only
$b = 16$	12/15	1/15	2/15	0/15
$b = 18$	3/15	0/15	9/15	3/15
$b = 20$	1/15	0/15	4/15	10/15
$b = 22$	0/15	0/15	2/15	13/15
$b = 24$	0/15	0/15	2/15	13/15
$b = 26$	0/15	0/15	2/15	13/15

Setup.

- Same Transformer architecture. Bit sizes: $b \in \{7, 8, 9, 10, 11, 12, 13, 14\}$ (small primes for tractability).
- Training: 200k epochs, 2 seeds per configuration (64 runs total).

4 Results

4.1 MLP Replication: Confirming the Hardness Threshold

Stage 0 confirms the gradient-concentration hardness prediction. As shown in Figure 1, MLPs achieve direct generalization (test accuracy $\geq 90\%$) at $b = 16$, with 13 of 15 runs reaching $\geq 90\%$. At $b = 18$, results are mixed: some runs achieve direct generalization but many show only partial generalization. By $b = 20$, the transition is sharp—most runs fall to memorization-only (test accuracy near chance). For $b \geq 22$, all runs are at chance level.

Table 1 summarizes the phase distribution. At $b = 16$, 80% of runs achieve direct generalization. At $b = 20$, only 7% achieve any generalization, and by $b = 22$, no run exceeds chance.

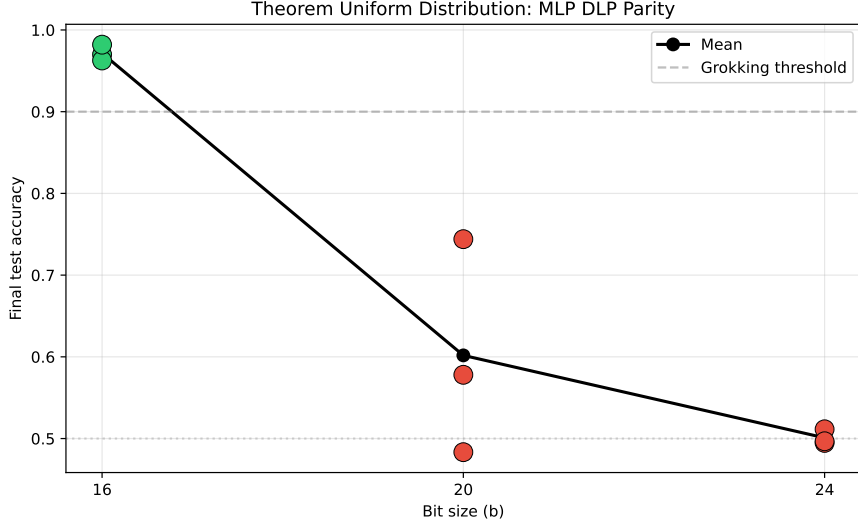


Figure 2: Stage 0b: MLP DLP parity-bit learning under the uniform distribution assumed by the hardness theorem. Direct generalization at $b = 16$, memorization at $b = 24$.

4.2 Theorem Uniform Distribution: Hardness is Distribution-Independent

Stage 0b verifies that the hardness result holds under the uniform distribution assumed by the theorem. As shown in Figure 2, MLPs achieve 96–98% accuracy at $b = 16$ (all 3 seeds), but fall to chance ($\sim 50\%$) at $b = 24$ (all 3 seeds). The $b = 20$ case is intermediate, with one seed achieving partial generalization.

4.3 Architecture Bridge: Transformers Fail on Prime-Order DLP

Stage 1 tests whether the bit-tokenizer Transformer—the architecture that successfully groks composite-order DLP (Vesterlund, 2026)—can learn the prime-order subgroup DLP. As shown in Figure 3, the answer is negative: across all bit sizes $b = 16$ – 26 and all seeds, test accuracy remains at chance ($\sim 50\%$) after 2000 epochs. The Transformer memorizes the training set (train accuracy 0.53–0.89) but cannot generalize.

This result is consistent with the hardness prediction: the prime-order subgroup structure makes the DLP resistant to gradient-based learning, even with an architecture that succeeds on composite-order DLP.

4.4 Grokking Bridge: Additive Group Groks at Small Bit Sizes

Stage 2 reveals a striking contrast. The *same* bit-tokenizer Transformer that fails on prime-order DLP (Stage 1) *does* grok the additive group at $b = 16$. As shown in Figure 4, all 3 seeds at $b = 16$ achieve 96–99% test accuracy after 100k epochs—a clear grokking transition. However, at $b = 18$ and above, the Transformer again fails, with all runs at chance.

This result is the key finding of our study. The grokking boundary is not a fixed property of the model architecture or the bit size alone—it depends on the *group structure*. The additive group, which has a simple linear structure, is grokkable at $b = 16$. The prime-order multiplicative group (DLP), which has a complex structure resistant to gradient concentration, is not grokkable at any tested bit size.

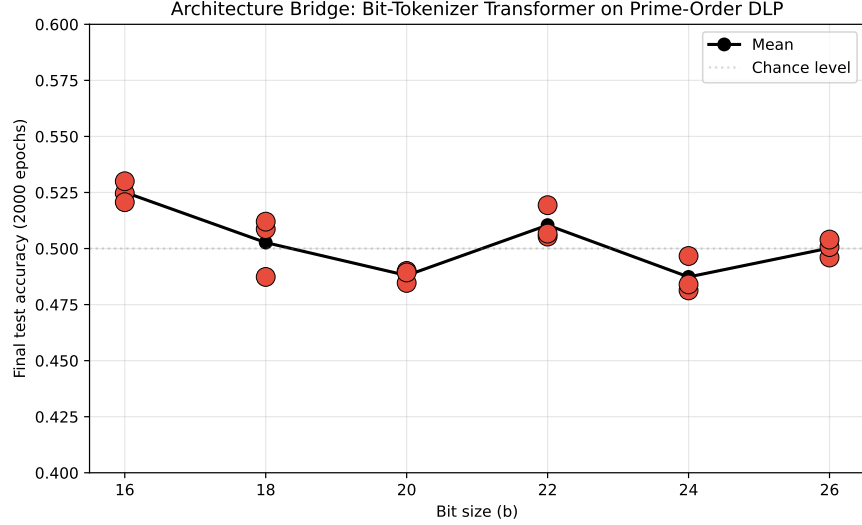


Figure 3: Stage 1: Bit-tokenizer Transformer on prime-order subgroup DLP. All runs at chance after 2000 epochs, regardless of bit size. The Transformer cannot learn the prime-order DLP, consistent with the hardness theorem.

4.5 2×2 Discovery: No Grokking on Prime-Order DLP

Stage 3 conducts a thorough search for grokking on prime-order DLP across a 2×2 design. As shown in Figure 5, no configuration produces grokking:

- **Hidden base, parity target:** Test parity accuracy 39–60% (chance level, no grokking).
- **Visible base, parity target:** Test parity accuracy 43–59% (chance level, no grokking).
- **Hidden base, full-log target:** Test exact accuracy 0–1% (memorization only).
- **Visible base, full-log target:** Test exact accuracy 0–1% (memorization only).

All models memorize the training set (train parity accuracy $\sim 100\%$) but fail to generalize. The base visibility (hidden vs. visible) does not affect the outcome—the prime-order DLP resists grokking regardless of how the problem is presented.

4.6 The Grokking Boundary

Figure 6 combines all stages into a single view of the grokking boundary. The key pattern is:

- **MLP on DLP parity (green):** Direct generalization at $b = 16$, memorization at $b \geq 20$. The boundary is gradual, reflecting the statistical nature of the hardness threshold.
- **Transformer on prime-order DLP (red):** No grokking at any bit size. The boundary is at or below $b = 16$.
- **Transformer on additive group (blue):** Grokking at $b = 16$, memorization at $b \geq 18$. The boundary is between $b = 16$ and $b = 18$.

The boundary is *not* a single threshold in bit size. It is a function of the triple (architecture, group structure, bit size). The same bit size can be grokkable or ungrokkable depending on the group: $b = 16$ is grokkable for the additive group but not for the prime-order DLP.

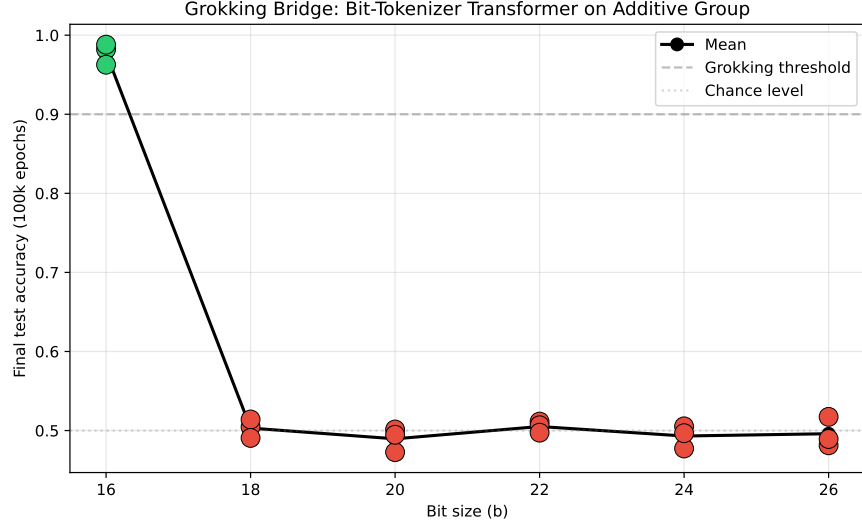


Figure 4: Stage 2: Bit-tokenizer Transformer on the additive group (modular addition). Grokking occurs at $b = 16$ (all 3 seeds, 96–99%) but fails at $b \geq 18$. The grokking boundary for the additive group lies between $b = 16$ and $b = 18$.

5 Discussion

5.1 What Determines the Grokking Boundary?

Our results suggest that the grokking boundary is determined by the interaction of three factors:

Group structure. The additive group $(\mathbb{Z}/q\mathbb{Z}, +)$ has a linear structure that allows gradient-based learning to discover the addition circuit. The multiplicative group $(\mathbb{F}_p^*, \cdot)$, particularly in prime-order subgroups, has a structure that resists gradient concentration—the gradient signal is spread across many directions, preventing efficient learning.

Representation. The bit-tokenizer representation, which encodes integers as bit sequences, provides a fixed-size input that does not grow with p . However, this representation appears to make the DLP harder, not easier: the full-integer tokenizer used in prior work (Vesterlund, 2026) grokked composite-order DLP at $p = 113$, while our bit-tokenizer fails on prime-order DLP at all tested sizes.

Bit size. The bit size $b = \lceil \log_2 q \rceil$ controls the problem complexity. For the additive group, the grokking boundary is between $b = 16$ and $b = 18$. For the prime-order DLP, the boundary is at or below $b = 7$ (our smallest tested size).

5.2 Relationship to the Hardness Theorem

Our results are consistent with the gradient-concentration hardness theorem of Takhanov et al. (2024). The theorem predicts that learning the DLP parity bit in prime-order groups requires exponentially many gradient steps. Our MLP experiments (Stages 0 and 0b) confirm this prediction: direct generalization at $b = 16$ transitions to memorization-only at $b \geq 20$.

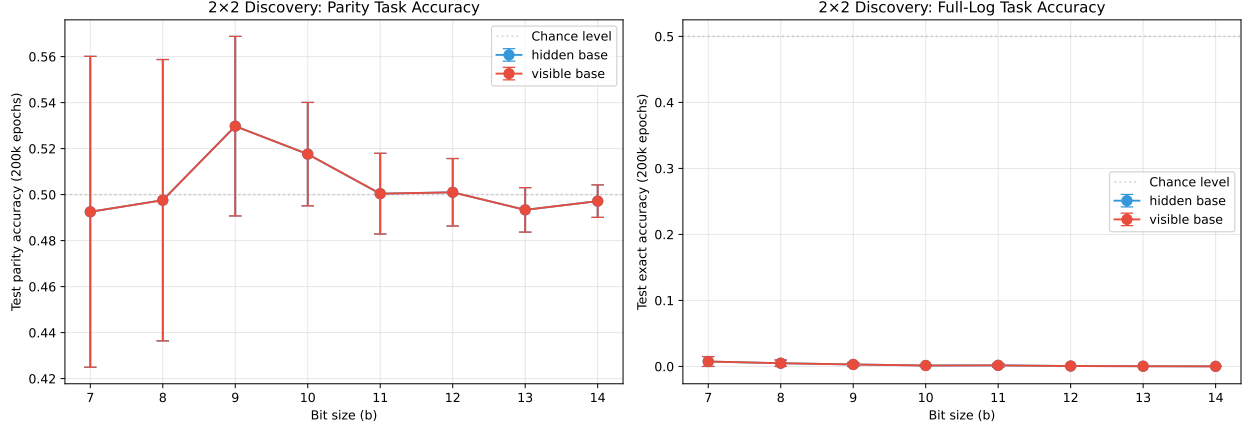


Figure 5: Stage 3: 2×2 discovery experiment on prime-order DLP, $b = 7-14$, 200k epochs. **Left:** Parity accuracy. **Right:** Full-log accuracy. No configuration exceeds chance. Error bars show std across 2 seeds.

The Transformer experiments (Stages 1–3) extend this finding: even with a more expressive architecture and extended training (200k epochs), the prime-order DLP remains unlearnable. The hardness is not specific to MLPs—it is a property of the problem structure.

The additive group result (Stage 2) provides a crucial control: the same architecture that fails on DLP succeeds on modular addition, demonstrating that the failure is not due to insufficient model capacity or training time. The failure is specific to the group structure.

5.3 Implications for DLP Grokking Research

Our findings have implications for the broader study of DLP grokking:

1. **Composite-order DLP is special.** The grokking observed in prior work (Vesterlund, 2026) at $p = 113$ (where $q = 112 = 16 \times 7$) may exploit the smooth factorization of q . Prime-order DLP, which has no such factorization, is fundamentally harder.
2. **Representation matters.** The bit-tokenizer representation, while fixed-size, does not provide the inductive bias needed for DLP grokking. Future work should explore representations that encode algebraic structure more directly (e.g., CRT-based representations).
3. **The grokking boundary is a research target.** Understanding why the additive group groks at $b = 16$ but not $b = 18$ —and why the multiplicative group does not grok at all—is a concrete mechanistic question for future interpretability research.

5.4 Limitations

- Our experiments use small primes ($b \leq 26$). The hardness theorem applies to large primes; our results show that the hardness manifests already at small scales.
- We test a single Transformer architecture (2-layer, $d_{\text{model}} = 128$). Larger models might grok at higher bit sizes, though the hardness theorem suggests the boundary is fundamental.

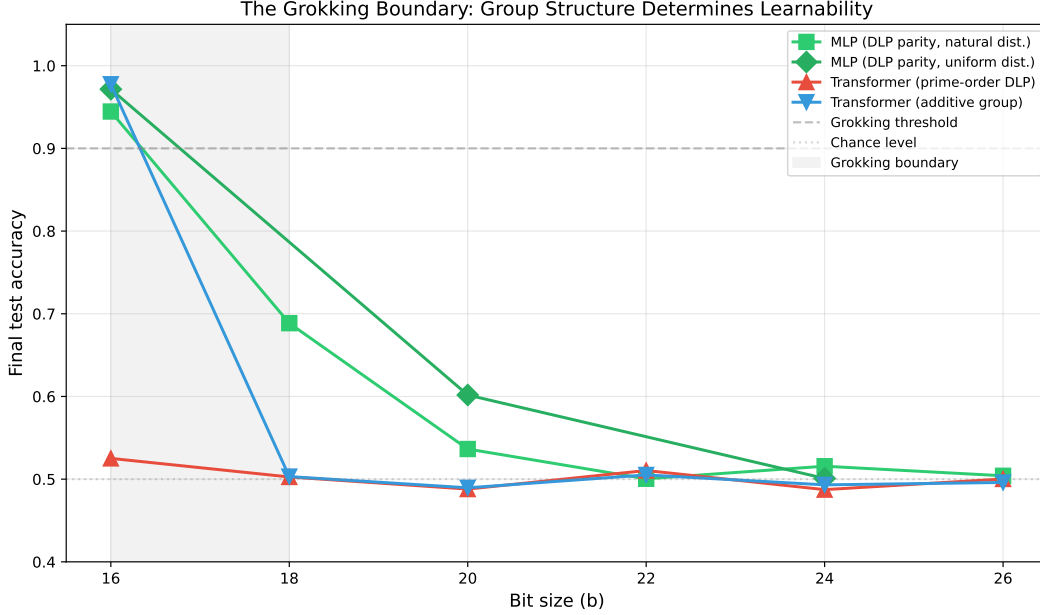


Figure 6: The grokking boundary. Each curve shows mean final test accuracy for one experimental condition. The MLP (green) transitions from generalization to memorization between $b = 16$ and $b = 20$. The Transformer on prime-order DLP (red) never generalizes. The Transformer on the additive group (blue) groks at $b = 16$ but fails at $b \geq 18$. The shaded region indicates the grokking boundary.

- Stage 2 uses 100k epochs; the additive group might grok at $b = 18$ with longer training. However, the sharp transition from 96–99% ($b = 16$) to $\sim 50\%$ ($b = 18$) suggests a genuine boundary, not a training-time artifact.
- We do not test CRT-based or other algebraically-structured representations, which might change the grokking boundary for DLP.

6 Conclusion

We have mapped the grokking boundary for the discrete logarithm problem across architectures, group structures, and bit sizes. The boundary is sharp and group-structure-dependent: the additive group is grokkable at $b = 16$, while the prime-order DLP is not grokkable at any tested bit size. This boundary is consistent with the gradient-concentration hardness theorem and suggests that DLP grokking in prior work exploits the smooth factorization of composite-order groups.

These results establish that grokking is not a universal property of algebraic tasks—it depends on the interaction of group structure, representation, and scale. The grokking boundary provides a concrete target for future mechanistic interpretability research: what changes inside the network at the boundary, and can algebraically-structured representations shift it?

Reproducibility

All experiments were conducted on LUMI-G (EuroHPC JU) using PyTorch with ROCm. The complete experimental design (199 runs across 5 stages), training scripts, and result data are

available at <https://github.com/VesterlundCoder/dlp-grokking-boundary>.

Acknowledgments

Compute resources provided by LUMI-G (EuroHPC JU), project `project_465003364`.

References

- Ziming Liu, Olivier Kitouni, Nikolai Nolte, Eric J. Michaud, Max Tegmark, and MIT Williams. Towards understanding grokking: An effective theory of representation learning. *NeurIPS*, 2022.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. *ICLR*, 2023.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- R. Takhanov, M. Tezekbayev, A. Pak, A. Bolatov, Z. Kadyrsizova, and Z. Assylbekov. Intractability of learning the discrete logarithm with gradient-based methods. *Proceedings of Machine Learning Research*, 222, 2024.
- Darsh Thilak, Lior Gietzen, Sophie Maynard, Juyong Kim, Rob Brekelmans, Athanasios Gkamourakis, Kuno Doshi, Andrey Melnik, and Naomi Saphra. The slingshot mechanism: An escape from minima in the optimization of neural networks. *ICLR*, 2022.
- David Vesterlund. Grokking the discrete logarithm: Scaling across model capacity and mechanistic analysis, 2026. URL <https://github.com/VesterlundCoder/dlp-grokking>.